Pre-Search and Discovery Document for LegacyLens

Building RAG Systems for Legacy Enterprise Codebases

Primary Codebase: GnuCOBOL (Open Source COBOL Compiler)

Date: March 2, 2026
Author: Luis Ramos
Project: GauntletAI — Week 3

Table of Contents

Phase 1: Define Your Constraints

1. Scale & Load Profile
2. Budget & Cost Ceiling
3. Time to Ship
4. Data Sensitivity
5. Team & Skill Constraints

Phase 2: Architecture Discovery

6. Vector Database Selection
7. Embedding Strategy
8. Chunking Approach
9. Retrieval Pipeline
10. Answer Generation
11. Framework Selection

Phase 3: Post-Stack Refinement

12. Failure Mode Analysis
13. Evaluation Strategy
14. Performance Optimization
15. Observability
16. Deployment & DevOps

Appendix A: Technology Decision Matrix

Appendix B: Cost Projection Tables

Executive Summary

LegacyLens is a RAG-powered system designed to make the GnuCOBOL open-source COBOL compiler codebase queryable and understandable through natural language. This Pre-Search document captures the structured discovery process completed before writing any code, covering constraints analysis, architecture decisions, and technology selections.

The GnuCOBOL project is a mature, open-source COBOL-to-C transpiler with approximately 241,000+ lines of code across hundreds of files. The codebase is primarily written in C (with Yacc/Bison grammars and Lex/Flex scanners) and includes COBOL test programs and sample utilities. This makes it an ideal target for a legacy code RAG system: it is large enough to be non-trivial, open source (GPLv3+/LGPLv3+), and representative of real enterprise COBOL infrastructure.

Phase 1: Define Your Constraints

1. Scale & Load Profile

How large is the target codebase (LOC, file count)?

GnuCOBOL v3.2 contains approximately 241,645 lines of analyzed C/C++ code (per Coverity Scan), with the compiler module (cobc/) accounting for ~128,000 LOC and the runtime library (libcob/) adding ~46,000 LOC. Including test files, configuration files, build scripts, documentation, and COBOL sample programs, the total file count reaches into the hundreds. Key files include:

Component	Approx. LOC	Key Files
Compiler (cobc/)	~128,000	parser.y (18K+ lines), codegen.c, typeck.c, tree.c
Runtime (libcob/)	~46,000	common.c, call.c, fileio.c, intrinsic.c, numeric.c
Test Suite	~50,000+	37,000+ tests across ~40 .at files, NIST COBOL 85 suite
Config/Dialect Files	~5,000	13+ COBOL dialect configs (.conf, .words, .ttbl)
Build System	~3,000	configure.ac, Makefile.am files, m4 macros
Documentation	~5,000	Texinfo (.texi), man pages
COBOL Samples	~2,000	extras/ directory, test COBOL programs (.cob, .cbl)

This comfortably exceeds the minimum requirement of 10,000+ LOC across 50+ files.

Expected query volume?

For the MVP and demonstration phase, query volume will be low — estimated at 10-50 queries per day during development and testing, and up to 200 queries/day during evaluation. In a production scenario with a team of developers onboarding to the GnuCOBOL codebase, this could scale to 500-1,000 queries/day. The system must handle concurrent queries from multiple users.

Batch ingestion or incremental updates?

Primary mode: Batch ingestion. The GnuCOBOL codebase will be ingested in full during initial setup. GnuCOBOL releases are infrequent (the latest stable release 3.2 was in July 2023), so incremental updates are a lower priority. The architecture should support re-ingestion of the full codebase when a new version is released, which should complete within the 5-minute performance target.

Latency requirements for queries?

Target: <3 seconds end-to-end (from query submission to answer display), as specified in the project performance targets. This includes query embedding (~100-200ms), vector similarity search (~50-100ms), context assembly (~50ms), and LLM answer generation (~1-2 seconds). The LLM generation step will be the bottleneck; streaming responses will improve perceived latency.

2. Budget & Cost Ceiling

Vector database hosting costs?

Target: $0/month for MVP using free tiers. Pinecone offers 2 GB free (sufficient for ~100K+ vectors at 1536 dimensions). Qdrant Cloud offers 1 GB free forever. Supabase (pgvector) offers 500 MB free. ChromaDB is entirely free as an embedded database. For production, budget up to $25/month (Supabase Pro or Qdrant Flex).

Embedding API costs (per token)?

Model	Cost per 1M Tokens	Est. Cost for Full Codebase (25M tokens)
OpenAI text-embedding-3-small	$0.02	$0.50
OpenAI text-embedding-3-large	$0.13	$3.25
Voyage Code 3	$0.18 (200M free tokens)	$0.00 (within free tier)
Cohere embed-english-v3	$0.10	$2.50
sentence-transformers (local)	Free (compute only)	~$0.50 GPU time

LLM API costs for answer generation?

Estimated at 500-1,000 tokens per answer generation call (prompt + retrieved context + response). Using Claude Sonnet 4: $3/$15 per 1M input/output tokens. At 200 queries/day with ~2,000 input tokens and ~500 output tokens per query: ~$0.12 + $0.15 = ~$0.27/day or ~$8.10/month. Using GPT-4o-mini at $0.15/$0.60 per 1M tokens: ~$0.06 + $0.06 = ~$0.12/day or ~$3.60/month.

Where will you trade money for time?

Prioritize managed services over self-hosting to maximize development velocity. Specifically: (1) Use a managed vector database (Pinecone or Supabase) instead of self-hosting Qdrant/Milvus — the free tiers eliminate cost while removing DevOps burden. (2) Use API-based embeddings (Voyage Code 3 or OpenAI) rather than self-hosting sentence-transformers — the quality difference and zero-infrastructure benefit justify the minimal cost. (3) Use Vercel for frontend deployment (free tier) to avoid configuring servers.

3. Time to Ship

MVP timeline?

24 hours from project start. The MVP must have: codebase ingested, syntax-aware chunking, embeddings generated and stored, semantic search working, a query interface (CLI or web), code snippets returned with file/line references, basic LLM-powered answer generation, and public deployment.

Which features are must-have vs nice-to-have?

Priority	Feature	Deadline
Must-Have (MVP)	Ingest GnuCOBOL codebase	24 hours
Must-Have (MVP)	Syntax-aware chunking	24 hours
Must-Have (MVP)	Embedding generation & vector storage	24 hours
Must-Have (MVP)	Semantic search across codebase	24 hours
Must-Have (MVP)	Natural language query interface	24 hours
Must-Have (MVP)	Code snippets with file/line references	24 hours
Must-Have (MVP)	LLM answer generation from context	24 hours
Must-Have (MVP)	Public deployment	24 hours
Must-Have (Final)	4+ Code Understanding Features	Final deadline
Must-Have (Final)	AI Cost Analysis document	Final deadline
Must-Have (Final)	RAG Architecture Documentation	Final deadline
Nice-to-Have	Hybrid search (vector + keyword)	If time permits
Nice-to-Have	Re-ranking pipeline	If time permits
Nice-to-Have	Multiple codebase support	If time permits
Nice-to-Have	Codebase visualization / dependency graphs	If time permits

Framework learning curve acceptable?

Yes, with caveats. LangChain and LlamaIndex both have learning curves but extensive documentation and examples. For the 24-hour MVP, a lightweight custom pipeline (direct API calls to embedding service + vector DB SDK) may be faster than learning a framework's

abstractions. The framework can be adopted for the final submission when there is more time for integration. Decision: start with a custom pipeline for MVP, consider LangChain integration for the final version if it adds value.

4. Data Sensitivity

Is the codebase open source or proprietary?

Fully open source. GnuCOBOL is licensed under GPLv3+ (compiler) and LGPLv3+ (runtime library). The source code is publicly available on SourceForge and GNU Savannah, with GitHub mirrors maintained by third parties (OCamlPro, paulsmith). There are zero restrictions on processing, embedding, or storing this code.

Can you send code to external APIs?

Yes, without restriction. Since GnuCOBOL is open source and publicly available, sending code snippets to OpenAI, Voyage AI, Anthropic, or any other external API for embedding or analysis raises no privacy, IP, or compliance concerns. This is a significant advantage over proprietary enterprise COBOL codebases where data residency and confidentiality would be critical constraints.

Data residency requirements?

None. The codebase is publicly available and not subject to data sovereignty regulations (GDPR, CCPA, etc. do not apply to open-source compiler source code). Vector databases and APIs can be hosted in any region. For production deployment targeting enterprise users with proprietary codebases, data residency would become a critical concern — this should be noted as a future consideration.

5. Team & Skill Constraints

Familiarity with vector databases?

Moderate. Experience with relational databases (PostgreSQL, MySQL) and NoSQL (MongoDB). Limited hands-on experience with dedicated vector databases (Pinecone, Qdrant). pgvector via Supabase leverages existing PostgreSQL knowledge and is the lowest-friction option. Pinecone's managed API is also straightforward. Self-hosting Milvus or Weaviate would require significant ramp-up time.

Experience with RAG frameworks (LangChain, LlamaIndex)?

Basic familiarity with LangChain concepts (chains, retrievers, document loaders). No production experience with LlamaIndex. For the MVP, a custom pipeline with direct SDK calls will be more predictable than debugging framework abstractions under time pressure. LangChain can be evaluated for the final submission.

Comfort with the target legacy language?

Limited direct COBOL programming experience. However, COBOL's verbose, English-like syntax makes it more readable than many languages. Key COBOL concepts are well-documented:

DIVISIONS (IDENTIFICATION, ENVIRONMENT, DATA, PROCEDURE), SECTIONS, PARAGRAPHS, COPY statements, PERFORM loops, and fixed-format column rules (columns 1-6: sequence numbers, column 7: indicator, columns 8-72: code area). The GnuCOBOL codebase itself is primarily C, which is familiar. Understanding COBOL structure is essential for designing effective chunking strategies.

Phase 2: Architecture Discovery

6. Vector Database Selection

Managed vs self-hosted?

Decision: Managed. Given the 24-hour MVP timeline, eliminating infrastructure management is critical. Managed options with free tiers (Pinecone, Qdrant Cloud, Supabase pgvector) provide sufficient capacity for the target codebase size while requiring zero DevOps effort.

Filtering and metadata requirements?

Essential. Each code chunk needs rich metadata for effective filtering:

Metadata Field	Example Values	Use Case
file_path	cobc/parser.y, libcob/fileio.c	Scope queries to specific modules
line_start / line_end	1450-1520	Enable precise source references
language	c, cobol, yacc, lex	Filter by file type
component	compiler, runtime, tests, config	Architectural-level filtering
chunk_type	function, struct, comment_block, config	Structural filtering
function_name	cb_build_move, cob_file_open	Direct function lookup
dependencies	['common.h', 'tree.h']	Dependency tracking

All candidate databases support metadata filtering. Supabase/pgvector offers the most flexible filtering via full SQL WHERE clauses. Pinecone and Qdrant have purpose-built metadata filter operators that are highly performant.

Hybrid search (vector + keyword) needed?

Highly desirable. Legacy code queries often mix semantic intent with exact identifiers. For example: "What does the cb_build_move function do?" requires semantic understanding of 'what does it do' combined with exact keyword matching on 'cb_build_move'. Similarly, COBOL-specific queries like "Find all PERFORM statements in the compiler" benefit from keyword matching on 'PERFORM'. Hybrid search support by candidate:

Database	Hybrid Search	Mechanism
Pinecone	Yes	Dense + sparse vectors, tunable alpha weight
Qdrant	Yes	Native sparse vector support, RRF fusion
ChromaDB	Partial	Separate vector/BM25 modes,

		app-level fusion
Weaviate	Best-in-class	Native BM25 + vector, built-in fusion
Supabase pgvector	Yes	tsvector (full-text) + pgvector, RRF fusion
Milvus/Zilliz	Yes	Dense + sparse + full-text search combined

Scaling characteristics?

At 10K-50K chunks with 1024-1536 dimensions, all candidates handle the load trivially. The scaling question becomes relevant if LegacyLens expands to index multiple codebases or very large enterprise repositories (millions of chunks). Pinecone and Milvus are built for billion-scale; Qdrant and Weaviate handle millions efficiently; ChromaDB and pgvector are best suited for hundreds of thousands.

Selection Decision

Primary Choice: Supabase pgvector

Rationale: (1) Unified data layer — vectors, metadata, user data, and application state in one database, reducing architectural complexity. (2) Hybrid search via tsvector + pgvector with Reciprocal Rank Fusion is well-documented and effective for code search. (3) Free tier (500 MB) is sufficient for the full GnuCOBOL codebase. (4) Full SQL power for metadata filtering surpasses dedicated vector DBs. (5) Familiar PostgreSQL interface reduces learning curve. (6) Supabase provides Auth, Edge Functions, and real-time capabilities for the application layer.

Fallback Choice: Pinecone

Rationale: If pgvector performance proves insufficient or hybrid search quality is inadequate, Pinecone offers a battle-tested managed service with excellent hybrid search (alpha-tunable dense + sparse vectors) and a generous free tier.

7. Embedding Strategy

Code-specific vs general-purpose model?

Decision: Code-specific. Voyage Code 3 outperforms OpenAI text-embedding-3-large by 13.8% on code retrieval benchmarks across 238 datasets. While no embedding model has been fine-tuned on COBOL specifically, code-specific models better capture structural patterns (function boundaries, control flow, data declarations) that appear across programming languages. COBOL's verbose, English-like syntax ("MOVE X TO Y", "PERFORM PARAGRAPH-NAME") actually benefits code-specific models that understand both natural language and code structure.

Dimension size tradeoffs?

Model	Dimensions	Storage per 50K	Quality

		Chunks	
Voyage Code 3 (default)	1024	~200 MB	Best for code
Voyage Code 3 (reduced)	512	~100 MB	Good (Matryoshka)
OpenAI text-embedding-3-small	1536	~300 MB	Good general
OpenAI text-embedding-3-large	3072	~600 MB	Better general

Decision: Voyage Code 3 at 1024 dimensions. This provides the best quality-to-storage ratio for code retrieval. The 200 MB storage footprint fits comfortably within free-tier limits of all candidate vector databases.

Local vs API-based embedding?

Decision: API-based (Voyage Code 3). The 200M free token allowance covers the entire GnuCOBOL codebase multiple times over (estimated 25M tokens for full ingestion). API-based embedding eliminates GPU infrastructure requirements, provides consistent quality, and is operationally simpler. Local models (sentence-transformers) would be free but offer significantly lower quality for code understanding and have restrictive context windows (256-512 tokens).

Batch processing approach?

Voyage Code 3 supports batch embedding with up to 128 texts per API call. Strategy: (1) Process all chunks into batches of 100-128. (2) Implement exponential backoff for rate limiting. (3) Store embeddings locally as a checkpoint before vector DB insertion to avoid re-embedding on upload failures. (4) Target ingestion throughput: 10,000+ LOC in <5 minutes (per performance target). With batch API calls, the full 50K-chunk codebase should embed in 2-3 minutes.

Selection Decision

Primary Choice: Voyage Code 3 (1024 dimensions)

Rationale: (1) Purpose-built for code retrieval — 13.8% better than OpenAI large. (2) 32K token context window — handles verbose COBOL procedures without truncation. (3) 200M free tokens — entire codebase embeds at zero cost. (4) Matryoshka support — can reduce to 512 dims if storage becomes a concern. (5) Quantization options (int8, binary) for further storage optimization.

Fallback Choice: OpenAI text-embedding-3-small (1536 dimensions)

Rationale: Ultra-low cost ($0.50 for full codebase), wide ecosystem support, and 8K context window is sufficient for 500-token chunks.

8. Chunking Approach

Syntax-aware vs fixed-size?

Decision: Hybrid approach — syntax-aware as the primary strategy with fixed-size + overlap as a fallback. The GnuCOBOL codebase contains multiple file types requiring different chunking strategies:

File Type	Strategy	Boundaries
C source (.c)	Function-level	Function definitions as natural boundaries
C headers (.h)	Struct/typedef-level	Type definitions, macro groups
Yacc/Bison (.y)	Rule-level	Grammar rules as semantic units
Lex/Flex (.l)	Pattern-level	Lexer rules grouped by category
COBOL (.cob, .cbl)	Paragraph/Section-level	COBOL PARAGRAPH and SECTION boundaries
Config (.conf, .def)	Block-level	Configuration blocks as units
Test files (.at)	Test-case-level	Individual test cases as chunks
Unstructured text	Fixed-size + overlap	500 tokens with 50-token overlap

Optimal chunk size for your embedding model?

Voyage Code 3 supports up to 32K tokens per input, giving significant flexibility. However, retrieval precision degrades with very large chunks (too much noise) and very small chunks (insufficient context). Target chunk sizes:

Chunk Type	Target Size	Rationale
Small function	100-300 tokens	Keep as single chunk, self-contained
Medium function	300-800 tokens	Ideal size for semantic search
Large function	800-1500 tokens	Split at logical sub-blocks if possible
Very large (parser.y rules)	500-1000 tokens	Split at grammar rule boundaries
Fallback fixed-size	500 tokens	50-token overlap for context continuity

Overlap strategy?

For syntax-aware chunks: Include 2-3 lines of surrounding context (the preceding comment block and function signature of the next function) to maintain continuity. For fixed-size fallback chunks: 50-token overlap (~10%) to prevent semantic breaks at chunk boundaries. For hierarchical chunks: parent chunk summary embedded as metadata in child chunks (e.g., file-level summary attached to each function chunk).

Metadata to preserve?

Each chunk will carry the following metadata for retrieval and display:

Metadata Field	Description	Source
file_path	Relative path from repo root	File system
line_start	Starting line number	Parser
line_end	Ending line number	Parser
language	Programming language	File extension
chunk_type	function, struct, rule, paragraph, etc.	Parser
name	Function/paragraph/rule name	Parser
parent_file_summary	1-sentence file description	LLM-generated or comment
dependencies	Included files, called functions	Static analysis
component	compiler, runtime, tests, config	Directory path

9. Retrieval Pipeline

Top-k value for similarity search?

Decision: k=10 for initial retrieval, narrowed to top-5 after re-ranking. Rationale: The project target is >70% relevant chunks in top-5. Retrieving 10 candidates gives the re-ranker enough material to find 5 truly relevant results. For broad queries ("show me error handling patterns"), k may be increased to 20.

Re-ranking approach?

Two-stage approach: (1) Initial retrieval from vector DB returns top-10 by cosine similarity. (2) Re-ranking using a cross-encoder or LLM-based scorer that considers the query-chunk pair holistically. For MVP, a simple heuristic re-ranker that boosts chunks matching exact keywords in the query. For the final version, consider Cohere Rerank or a lightweight cross-encoder (ms-marco-MiniLM-L-6-v2). If using Supabase hybrid search, the RRF fusion of semantic + keyword scores serves as a built-in re-ranking mechanism.

Context window management?

LLM context is limited (Claude Sonnet: 200K tokens, GPT-4o: 128K). Strategy: (1) Pack top-5 retrieved chunks into the prompt (~2,500-5,000 tokens). (2) Include file-level context summaries for each chunk's source file (~500 tokens). (3) Reserve ~1,000 tokens for the system prompt and query. (4) Total context per query: ~4,000-6,500 tokens — well within any LLM's limits. (5) For "drill down" queries, include the full file content (up to 10K tokens).

Multi-query or query expansion?

Nice-to-have for final version. Strategy: For ambiguous queries, generate 2-3 alternative phrasings using the LLM (e.g., "What does cb_build_move do?" → ["cb_build_move function implementation", "MOVE statement code generation", "data movement in compiler"]). Retrieve for each variant and merge results using Reciprocal Rank Fusion. For MVP, single-query retrieval is sufficient.

10. Answer Generation

Which LLM for synthesis?

Decision: Claude Sonnet 4 (primary) with GPT-4o-mini as fallback.

Model	Input Cost/1M	Output Cost/1M	Context Window	Strengths
Claude Sonnet 4	$3.00	$15.00	200K tokens	Excellent at code explanation, strong reasoning
GPT-4o-mini	$0.15	$0.60	128K tokens	Very cheap, good enough for simple queries
GPT-4o	$2.50	$10.00	128K tokens	Strong code understanding
Llama 3 70B (local)	Free (compute)	Free (compute)	8K tokens	No API costs, limited context

Claude Sonnet 4 offers the best balance of code understanding quality and cost for this use case. Its 200K context window provides headroom for including extensive code context. GPT-4o-mini can be used for simpler queries to reduce costs.

Prompt template design?

The prompt template will follow a structured format:

System Prompt: "You are LegacyLens, an expert assistant for understanding the GnuCOBOL codebase. You analyze code snippets and explain their purpose, behavior, and relationships. Always reference specific file paths and line numbers. If the retrieved context does not contain enough information to answer confidently, say so."

User Prompt: "Question: {user_query}\n\nRelevant Code

Context:\n{chunk_1_with_metadata}\n{chunk_2_with_metadata}\n...\n\nProvide a clear, specific answer referencing the code above."

Citation/reference formatting?

Every answer will include source references in the format: [file_path:line_start-line_end]. For example: "The cb_build_move function (cobc/typeck.c:2450-2520) validates data type compatibility before generating a MOVE operation." The UI will render these as clickable links that expand to show the full code context.

Streaming vs batch response?

Decision: Streaming. LLM answer generation is the slowest step (~1-2 seconds). Streaming tokens as they are generated significantly improves perceived latency. The web interface will display retrieved code snippets immediately upon retrieval completion, then stream the LLM explanation below them. This gives users useful information (the code) within ~500ms while the explanation streams in.

11. Framework Selection

LangChain vs LlamaIndex vs custom?

Framework	Pros	Cons	Fit
LangChain	Flexible, many integrations, good docs, large community	Abstraction overhead, breaking API changes, can be verbose	Good
LlamaIndex	Document-focused, great for structured data, built-in evaluation	Less flexible for custom pipelines, smaller community	Moderate
Haystack	Production-ready, evaluation tools, pipeline DSL	Steeper learning curve, less community traction	Moderate
Custom Pipeline	Full control, no abstraction overhead, minimal dependencies	More boilerplate, no built-in evaluation, reinventing wheels	Best for MVP

Decision: Custom pipeline for MVP, evaluate LangChain for final version. Rationale: The 24-hour MVP timeline favors directness over abstraction. A custom pipeline with direct SDK calls (Voyage AI SDK → Supabase client → Anthropic SDK) is predictable and debuggable. For the final version, LangChain's retriever abstractions and evaluation tools may add value.

Evaluation and observability needs?

Essential for measuring retrieval precision (>70% target). Tools to evaluate: (1) RAGAS — open-source RAG evaluation framework measuring faithfulness, answer relevancy, context precision, and context recall. (2) Manual evaluation — curate a test set of 20-30 queries with expected

results, score retrieval quality. (3) Logging — record every query, retrieved chunks, and generated answer for offline analysis.

Integration requirements?

The framework must integrate with: (1) Voyage AI API for embeddings, (2) Supabase for vector storage and metadata, (3) Anthropic API for answer generation, (4) Next.js/React frontend for the query interface, (5) Vercel for deployment. All of these have Python and/or JavaScript SDKs with well-documented APIs.

Phase 3: Post-Stack Refinement

12. Failure Mode Analysis

What happens when retrieval finds nothing relevant?

Scenario: User asks about something not in the codebase, or the query is too vague. Strategy: (1) Set a minimum similarity threshold (e.g., cosine similarity > 0.3). If no chunks exceed this threshold, return a message: "I couldn't find relevant code for your query. Try rephrasing or being more specific about the component you're interested in." (2) Suggest related topics based on the closest (but below-threshold) matches. (3) Offer example queries that work well. (4) Log these failures for analysis — they reveal gaps in chunking or embedding coverage.

How to handle ambiguous queries?

Strategy: (1) For very short queries (1-2 words), ask for clarification: "Could you provide more context? For example: 'What does the MOVE statement do?' or 'Where is MOVE implemented in the compiler?'" (2) For queries that match multiple distinct topics, present categorized results: "I found relevant code in both the compiler (cobc/) and runtime (libcob/). Which would you like to explore?" (3) Use query expansion (multi-query) to cover multiple interpretations.

Rate limiting and error handling?

External API dependencies introduce failure points: (1) Embedding API rate limits — implement exponential backoff with jitter, cache embeddings for repeated queries. (2) LLM API rate limits — queue requests, fallback to GPT-4o-mini if primary LLM is rate-limited. (3) Vector DB downtime — cache recent query results, return cached answers for repeated queries. (4) Network failures — return retrieved code snippets without LLM explanation as a degraded-but-useful response.

13. Evaluation Strategy

How to measure retrieval precision?

Precision@k metric: For each test query, manually label the top-k retrieved chunks as relevant or irrelevant. Precision@5 = (relevant chunks in top 5) / 5. Target: >70% (i.e., at least 3.5 out of 5 chunks are relevant on average). Also measure: Mean Reciprocal Rank (MRR) — how high does the first relevant result appear? NDCG@5 — are the most relevant results ranked highest?

Ground truth dataset for testing?

Create a test set of 20-30 queries covering the scenarios from the project spec:

Query	Expected Result (File/Function)	Category
Where is the main entry point?	cobc/cobc.c:main()	Navigation
What functions handle MOVE statements?	cobc/typeck.c:cb_build_move()	Function search
How does file I/O work at	libcob/fileio.c:cob_file_open()	Explanation

runtime?		
Find all PERFORM-related code	cobc/parser.y, cobc/codegen.c	Pattern search
What are the dependencies of the parser?	cobc/parser.y includes	Dependency
Show me error handling patterns	libcob/common.c error handling	Pattern search
Explain the code generation process	cobc/codegen.c	Explanation
What COBOL dialects are supported?	config/*.conf files	Configuration

Each query will have 3-5 manually identified relevant chunks. This enables automated precision measurement after each chunking or embedding strategy change.

User feedback collection?

Implement thumbs-up/thumbs-down on each answer. Store feedback with the query, retrieved chunks, and generated answer. Use negative feedback to identify: (1) Chunking gaps — relevant code not chunked properly. (2) Embedding failures — semantically relevant code not retrieved. (3) Generation errors — LLM misinterpreting retrieved context.

14. Performance Optimization

Caching strategy for embeddings?

Query embedding cache: Store a mapping of query text → embedding vector in-memory (LRU cache with 1,000-entry limit). This eliminates redundant API calls for repeated or similar queries. Chunk embeddings are computed once during ingestion and stored in the vector database — no runtime cache needed.

Index optimization?

For Supabase pgvector: Create an HNSW index on the embedding column with appropriate parameters (m=16, ef_construction=64 for 50K vectors). HNSW provides ~95-99% recall with sub-10ms query times at this scale. Also create standard B-tree indexes on metadata columns (file_path, language, component) for efficient filtered queries. For full-text search, create a GIN index on the tsvector column.

Query preprocessing?

Before embedding a query: (1) Normalize COBOL-specific terms (e.g., expand abbreviations: "FD" → "File Description", "PIC" → "PICTURE clause"). (2) Extract identifiers that should be matched exactly (function names, file paths). (3) Detect query type (navigation, explanation, search, dependency) to adjust retrieval parameters (e.g., higher k for search queries, lower k for navigation).

15. Observability

Logging for debugging retrieval issues?

Log every query with: (1) Raw query text. (2) Preprocessed query. (3) Query embedding vector (or hash). (4) Top-10 retrieved chunk IDs with similarity scores. (5) Re-ranked top-5 chunk IDs. (6) Generated answer. (7) User feedback (if given). (8) Total latency breakdown (embedding time, search time, generation time). Store logs in Supabase for easy querying and analysis.

Metrics to track (latency, precision, usage)?

Metric	Target	Collection Method
End-to-end latency	<3 seconds	Timer around full pipeline
Embedding latency	<200ms	Timer around API call
Search latency	<100ms	Timer around vector DB query
LLM generation latency	<2 seconds	Timer around API call (time to first token)
Retrieval precision@5	>70%	Manual evaluation on test set
Daily query volume	Track growth	Counter in logs
Error rate	<1%	Count failed queries / total queries
User satisfaction	>80% thumbs up	Feedback collection

Alerting needs?

For MVP: No automated alerting — manual log review. For production: Alert on (1) API error rate > 5% over 5 minutes, (2) Average latency > 5 seconds over 10 minutes, (3) Vector DB connection failures, (4) Embedding API quota approaching limits (>80% of monthly allocation).

16. Deployment & DevOps

CI/CD for index updates?

Strategy: (1) GitHub Actions workflow triggered manually or on new GnuCOBOL release tags. (2) Workflow steps: clone GnuCOBOL repo → run chunking pipeline → generate embeddings → upsert into vector database → run evaluation test suite → report precision metrics. (3) If precision drops below 70% on the test set, the pipeline fails and alerts for manual review. For MVP, ingestion will be run manually via a script.

Environment management?

Three environments: (1) Local development — ChromaDB embedded + local LLM for rapid iteration without API costs. (2) Staging — Supabase project (free tier) with Voyage/Anthropic APIs, deployed to Vercel preview. (3) Production — same Supabase project, deployed to Vercel

production. Environment variables managed via Vercel environment settings and .env.local files (excluded from git).

Secrets handling for API keys?

API keys required: (1) VOYAGE_API_KEY — for Voyage Code 3 embeddings. (2) ANTHROPIC_API_KEY — for Claude answer generation. (3) SUPABASE_URL + SUPABASE_ANON_KEY — for vector database access. (4) SUPABASE_SERVICE_ROLE_KEY — for server-side ingestion scripts (never exposed to client). All secrets stored in Vercel environment variables (encrypted at rest). Local development uses .env.local with .gitignore protection. No secrets committed to version control.

Appendix A: Technology Decision Matrix

Summary of all technology decisions with rationale:

Layer	Decision	Alternative	Rationale
Target Codebase	GnuCOBOL v3.2	OpenCOBOL Contrib	Larger codebase (241K+ LOC), real compiler, rich structure
Vector Database	Supabase pgvector	Pinecone	Unified data layer, SQL filtering, free tier, hybrid search
Embedding Model	Voyage Code 3 (1024d)	OpenAI text-embedding-3-small	Code-optimized, 32K context, 200M free tokens
LLM (Generation)	Claude Sonnet 4	GPT-4o-mini	Best code explanation quality, 200K context window
Chunking	Syntax-aware + fallback	Fixed-size only	Respects code boundaries, better retrieval precision
Framework	Custom pipeline (MVP)	LangChain	Faster to ship, no abstraction overhead under time pressure
Backend	Python/FastAPI or Next.js API routes	Node.js/Express	Python for ML/embedding libraries; Next.js if full-stack JS preferred
Frontend	Next.js + React	CLI only	Web UI with syntax highlighting, expandable results, streaming
Deployment	Vercel + Supabase	Railway	Free tiers, excellent DX, built-in CI/CD

Appendix B: Cost Projection Tables

Development & Testing Costs (Estimated)

Category	Service	Estimated Cost
Embedding (ingestion)	Voyage Code 3 — ~25M tokens	$0.00 (within 200M free tier)
Embedding (query dev/test)	Voyage Code 3 — ~1M tokens	$0.00 (within free tier)
LLM (development queries)	Claude Sonnet 4 — ~500 queries	~$5.00
Vector DB hosting	Supabase Free Tier	$0.00
Frontend hosting	Vercel Free Tier	$0.00
Total Development Spend		~$5.00

Production Cost Projections

Assumptions: 5 queries per user per day, ~2,500 input tokens and ~500 output tokens per query (including retrieved context), embedding cost for query only (~100 tokens per query), monthly vector DB storage stable at ~200 MB.

Cost Component	100 Users	1,000 Users	10,000 Users	100,000 Users
Embedding (queries)	$0.09/mo	$0.90/mo	$9.00/mo	$90.00/mo
LLM Generation (Claude Sonnet)	$52.50/mo	$525/mo	$5,250/mo	$52,500/mo
LLM Generation (GPT-4o-mini alt.)	$2.81/mo	$28.13/mo	$281.25/mo	$2,812.50/mo
Vector DB (Supabase)	$0 (free)	$25/mo (Pro)	$75/mo (Team)	$599/mo (Team)
Frontend Hosting (Vercel)	$0 (free)	$20/mo (Pro)	$20/mo (Pro)	Custom
Total (Claude Sonnet)	~$53/mo	~$571/mo	~$5,354/mo	~$53,189/mo
Total (GPT-4o-mini)	~$3/mo	~$74/mo	~$385/mo	~$3,502/mo

Key insight: LLM generation cost dominates at scale. Strategies to reduce: (1) Cache frequent query-answer pairs. (2) Use GPT-4o-mini for simple queries, reserve Claude for complex explanations. (3) Implement query classification to route appropriately. (4) Consider self-hosted LLMs (Llama 3) at 10K+ user scale.

Final: Selected Technology Stack

Layer	Technology
Primary Codebase	GnuCOBOL v3.2 (COBOL compiler, ~241K LOC, GPLv3+/LGPLv3+)
Vector Database	Supabase pgvector (managed PostgreSQL with vector extension)
Embedding Model	Voyage Code 3 (1024 dimensions, 32K context window)
LLM for Generation	Claude Sonnet 4 (primary), GPT-4o-mini (fallback/cost optimization)
Chunking Strategy	Syntax-aware (function/rule/paragraph) + fixed-size fallback
RAG Framework	Custom pipeline (MVP) → evaluate LangChain for final
Backend	Next.js API Routes or Python/FastAPI
Frontend	Next.js + React with syntax highlighting
Deployment	Vercel (frontend/API) + Supabase (database/vectors)
Search Type	Hybrid (semantic via pgvector + keyword via tsvector, RRF fusion)

— End of Pre-Search and Discovery Document —